Lecture 3
Arrays

15-122: Principles of Imperative Computation (Fall 2025)
Frank Pfenning, André Platzer

So far we have seen how to process primitive data like integers in imperative programs. That is useful, but certainly not sufficient to handle bigger amounts of data. In many cases we need *aggregate data structures* which contain other data. A common data structure, in particular in imperative programming languages, is that of an *array*. An array can be used to store and process a fixed number of data elements that all have the same type.

We will also take a first detailed look at the issue of program *safety*. A program is *safe* if it will execute without exceptional conditions which would cause its execution to abort. So far, only division and modulus are potentially unsafe operations, since division or modulus by 0 is defined as a runtime error.[1] Trying to access an array element for which no space has been allocated is a second form of runtime error. Array accesses are therefore potentially unsafe operations and must be proved safe.

Additional Resources

- Review slides (`https://cs.cmu.edu/~15122/handouts/slides/review/03-arrays.pdf`)
- Code for this lecture (`https://cs.cmu.edu/~15122/handouts/code/03-arrays.tgz`)

With respect to our learning goals we will look at the following notions.

Computational Thinking: Developing contracts that establish the safety of imperative programs.

Developing and evaluating proofs of the safety of code with contracts.

Programming: Identifying, describing, and effectively using arrays and **`for`**-loops.

[1] Another runtime error is for division or modulus of the smallest integer by -1.

 © Carnegie Mellon University 2025

1 Using Arrays

When t is a type, then t`[]` is the type of an array with elements of type t. Note that t is arbitrary: we can have an array of integers (**`int`**`[]`), and an array of booleans (**`bool`**`[]`) or an array of arrays of characters (**`char`**`[][]`). This syntax for the type of arrays is like Java, but is a minor departure from C, as we will see later in class.

Each array has a fixed size, and it must be explicitly allocated using the expression **`alloc_array`**`(t, n)`. Here t is the type of the array elements, and n is their number. With this operation, C0 will reserve a piece of memory with n elements, each having type t. Let's try in coin:

```
% coin
C0 interpreter (coin) 0.3.3 'Nickel'
Type '#help' for help or '#quit' to exit.
--> int[] A = alloc_array(int, 10);
A is 0xECE2FFF0 (int[] with 10 elements)
-->
```

The result may be surprising: A is an array of integers with 10 elements (obvious), but what does it mean to say A is `0xECE2FFF0` here? As we discussed in the lecture on integers, variables can only hold values of a small fixed size, the *word size* of the machine. An array of 10 integers would be 10 times this size, so we cannot hold it directly in the variable A. Instead, the variable A holds the *address* in memory where the actual array elements are stored. In this case, the address happens to be `0xECE2FFF0` (incidentally presented in hexadecimal notation), but there is no guarantee that the next time you run `coin` you will get the same address. Fortunately, this is okay because you cannot actually ever do anything directly with this address as a number and never need to either. Instead you access the array elements using the syntax `A[i]` where $0 \leq i < n$, where n is the length of the array. That is, `A[0]` will give you element 0 of the array, `A[1]` will be element 1, and so on. We say that arrays are *zero-based* because elements are numbered starting at 0. For example:

```
--> A[0];
0 (int)
--> A[1];
0 (int)
--> A[2];
0 (int)
--> A[10];
Error: accessing element 10 in 10-element array
```

```
Last position: <stdio>:1.1-1.6
--> A[-1];
Error: accessing negative element in 10-element array
Last position: <stdio>:1.1-1.6
-->
```

We notice that after allocating the array, all elements appear to be 0. This is guaranteed by the implementation, which initializes all array elements to a default value which depends on the type. The default value of type **`int`** is 0. Generally speaking, one should try to avoid exploiting implicit initialization because for a reader of the program it may not be clear if the initial values are important or not.

We also observe that trying to access an array element not in the specified range of the array will lead to an error. In this example, the valid accesses are `A[0]`, `A[1]`, ..., `A[9]` (which comes to 10 elements); everything else is illegal. And every other attempt to access the contents of the array would not make much sense, because the array has been allocated to hold 10 elements. How could we ever meaningfully ask what its element number 20 is if it has only 10? Nor would it make sense to ask `A[-4]`. In both cases, `coin` and `cc0` will give you an error message telling you that you have accessed the array outside the bounds. While an error is guaranteed in C0, in C no such guarantee is made. Accessing an array element that has not been allocated leads to *undefined behavior* and, in principle, anything could happen. This is highly problematic because implementations typically choose to just read from or write to the memory location where some element *would* be if it had been allocated. Since it has not been, some other unpredictable memory location may be altered, which permits infamous *buffer overflow attacks* which may compromise your machines.

How do we change an element of an array? We can use it on the left-hand side of an assignment. We can set `A[i] = e;` as long as e is an expression of the right type for an array element. For example:

```
--> A[0] = 5; A[1] = 10; A[2] = 20;
A[0] is 5 (int)
A[1] is 10 (int)
A[2] is 20 (int)
-->
```

After these assignments, the contents of memory might be displayed as follows, where $A =$ `0xECE2FFF0`:

				ECE30000					
0xECE2FFF0	F4	F8	FC		04	08	0C	10	14
5	**10**	**20**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]

Recall that an assignment (like `A[0] = 5;`) is a statement and as such has an effect, but no value. `coin` will print back the effect of the assignment. Here we have given three statements together, so all three effects are shown. Again, exceeding the array bounds will result in an error message and the program aborts, because it does not make sense to store data in an array at a position that is outside the size of that array.

```
--> A[10] = 100;
Error: accessing element 10 in 10-element array
Last position: <stdio>:1.1-1.6
-->
```

2 Using For-Loops to Traverse Arrays

A common pattern of access and traversal of arrays is for-loops, where an index i is counted up from 0 to the length of the array. To continue the example above, we can assign i^3 to the i-th element of the array as follows:

```
--> for (int i = 0; i < 10; i++)
... A[i] = i * i * i;
--> A[6];
216 (int)
-->
```

Characteristically, the exit condition of the loop tests for `i < n` where i is the array index and n is the length of the array (here 10).

After we type in the first line (the header of the for-loop), `coin` responds with the prompt `...` instead of `-->`. This indicates that the expression or statement it has parsed so far is incomplete. We complete it by supplying the body of the loop, the assignment `A[i] = i * i * i;`. Note that no assignment effect is printed. This is because the assignment is part of a loop. In general, `coin` will only print effects of top-level statements such as assignments, because when a complicated program is executed, a huge number of effects could be taking place.

3 Specifications for Arrays

When we use loops to traverse arrays, we need to make sure that all the array accesses are in bounds. In many cases this is evident, but it can be tricky in particular if we have two-dimensional data (for example, images). As an aid to this reasoning, we state an explicit loop invariant which expresses what will be true on every iteration of the loop.

To illustrate arrays, we will expand on our previous example, filling an array with cubes.

```
int[] cubes(int n) {
  int[] A = alloc_array(int, n);

  for (int i = 0; i < n; i++) {
    A[i] = i * i * i;
  }

  return A;
}
```

This looks straightforward. Is there a problem with the code or will it run correctly? In order to understand whether this function works correctly, we systematically develop a specification for it.

The first problem is the safety of the call to **`alloc_array`**, because allocating an array will fail if we ask for a negative number of elements. Since the number of elements we ask for in **`alloc_array(int, n)`** is n, and n is a parameter passed to the function, we need to add $n \geq 0$ into the precondition of the function.

For referring to the length of an array, C0 contracts have a special function `\length(A)` that stands for the number of elements in the array `A`. Just like the `\result` variable, the function `\length` is part of the contract language and cannot be used in C0 program code. Its purpose is to be used in contracts to specify the requirements and behavior of a program. For the cubes function, we want to specify the post-condition that the length of the array that the function returns is n.

```
int[] cubes(int n)
//@requires n >= 0;
//@ensures \length(\result) == n;
{
  int[] A = alloc_array(int, n);

  for (int i = 0; i < n; i++) {
    A[i] = i * i * i;
```

```
  }

  return A;
}
```

4 Loop Invariants for Arrays

By writing specifications, we should convince ourselves that all array accesses will be within the bounds. In the loop, we access `A[i]`, which would raise an error if `i` were negative or greater than or equal to `\length(A)`, because that would violate the bounds of the array.

Because `n` is not modified by the loop, we can use our knowledge that `\length(A) == n` from `A`'s declaration in conjunction with the loop guard `i < n` to conclude that `i` does not violate the upper bound (i.e., that `i < \length(A)` in each iteration of the loop). For the lower bound, we need to specify a loop invariant that ensures $i \geq 0$.

```
  for (int i = 0; i < n; i++)
  //@loop_invariant 0 <= i;
  {
    A[i] = i * i * i;
  }
```

Operationally, of course, we can reason that because `i` starts at 0 and only increments on every iteration, `i` can't ever be negative. But in this course we eschew such operational reasoning, instead encoding this information in loop invariant. We know that the loop invariant is true initially by the for loop's declaration: `i` is initially 0, and $0 \leq 0$. We furthermore know that, in an arbitrary iteration of the loop that initially $i < n$ by the loop guard, so $i' = i + 1$ cannot overflow to a negative number and the loop invariant is always preserved.

5 Aliasing

We have seen assignments to array elements, such as `A[0] = 0;`. But we have also seen assignments to array variables themselves, such as

```
int[] A = alloc_array(int, n);
```

What do they mean? To explore this, we separate the declaration of array variables (here: F and G) from assignments to them.

```
% coin -d cubes.c0
```

```
C0 interpreter (coin) 0.3.3 'Nickel'
Type '#help' for help or '#quit' to exit.
--> int[] F;
--> int[] G;
--> F = cubes(15);
F is 0xF6969A80 (int[] with 15 elements)
--> G[2];
Error: uninitialized value used
Last position: <stdio>:1.1-1.5
--> G = F;
G is 0xF6969A80 (int[] with 15 elements)
--> G = cubes(10);
G is 0xF6969A30 (int[] with 10 elements)
-->
```

The first assignment to `F` is as expected: it is the address of an array with 15 elements. The use of `G` in `G[2]`, of course, cannot succeed, because we have only declared `G` to have a type of integer arrays, but did not assign any array to `G`.

Afterward, however, when we assign `G = F`, then `G` and `F` (as locals) *hold the same address*! Holding the same address means that `F` and `G` are *aliased*. When we make the second assignment to `G` (changing its value) we get a new array, which is in fact smaller and definitely no longer aliased to `F` (note the different address). Aliasing (or the lack thereof) is crucial, because modifying one of two aliased arrays will also change the other. For example:

```
% coin
C0 interpreter (coin) 0.3.3 'Nickel'
Type '#help' for help or '#quit' to exit.
--> int[] A = alloc_array(int, 5);
A is 0xE8176FF0 (int[] with 5 elements)
--> int[] B = A;
B is 0xE8176FF0 (int[] with 5 elements)
--> A[0] = 42;
A[0] is 42 (int)
--> B[0];
42 (int)
-->
```

C0 has no built-in way to copy from one array to another (ultimately we will see that there are multiple meaningful ways to copy arrays of more complicated types). Here is a simple function to copy arrays of integers.

```
int[] array_copy(int[] A, int n)
//@requires n == \length(A);
//@ensures \length(\result) == n;
{
  int[] B = alloc_array(int, n);

  for (int i = 0; i < n; i++)
  //@loop_invariant 0 <= i;
  {
    B[i] = A[i];
  }

  return B;
}
```

For example, we can create B as a copy of A, and now assigning to the copy of B will not affect A. We will invoke coin with the `-d` flag to make sure that if a pre- or post-condition or loop invariant is violated we get an error message.

```
% coin copy.c0 -d
C0 interpreter (coin) 0.3.3 'Nickel'
Type '#help' for help or '#quit' to exit.
--> int[] A = alloc_array(int, 10);
A is 0xF3B8DFF0 (int[] with 10 elements)
--> for (int i = 0; i < 10; i++) A[i] = i*i;
--> int[] B = array_copy(A, 10);
B is 0xF3B8DFB0 (int[] with 10 elements)
--> B[9];
81 (int)
--> A[9] = 17;
A[9] is 17 (int)
--> B[9];
81 (int)
-->
```

6 Implementation Note

Internally, arrays are stored in the area of the memory we will call *allocated memory*. Memory in this area is allocated with **`alloc_array`**, which returns the address of an array (and later in this course **`alloc`**). In C0, memory is not explicitly deallocated, but it is *garbage collected* in the sense that memory

that can no longer be accessed from within the running program is freed so that it can be used to satisfy future allocation requests.

In order to check whether array accesses are in bounds, the C0 runtime system must store not only the array data, but also the length of the array. In the running program, this information cannot be accessed directly: given an array we cannot obtain its length. This is mostly in order to simulate safe programming practices in C. For example, when arrays are passed as arguments to functions we usually also pass (a bound on) their length.

However, in *contracts* (that is, function preconditions `@requires`, post-conditions `@ensures`, loop invariants `@loop_invariant` and C0 assertions `@assert`) we can refer to the length of an array using the special function `\length`. We have already used this in the examples above. For example, the copy function

```
int[] array_copy(int[] A, int n)
 //@requires n == \length(A);
 //@ensures \length(\result) == n;
 ;
```

requires `n` to be the length of the parameter array `A` and ensures that the result array will have length `n`.

7 Exercises

Exercise 1 (sample solution on page 12)**.** *Write a function* `array_part` *that creates a copy of a part of a given array, namely the elements from position* `lo` *(included) to position* `hi` *(excluded). Your function should have prototype*

```
int[] array_part(int[] A, int lo, int hi);
```

Develop a specification and loop invariants for this function. Prove that it works correctly by checking the loop invariant and proving array bounds.

Exercise 2 (sample solution on page 16)**.** *Write a function* `copy_into` *that copies a part of a given array* `source`*, namely* `n` *elements starting at position* `i`*, into another given array* `target`*, starting at position* `j`*. Your function should have prototype*

```
int copy_into(int[] source, int i, int n, int[] target, int j);
```

As an extra service, make your function return the last position in the target array that it entered data into, or `-1` *if no such position exists. Develop a specification and loop invariants for this function. Prove that it works correctly by checking the loop invariant and proving array bounds.*

Exercise 3 (sample solution on page 18)**.** *Can you develop a reasonable (non-degenerate) and useful function with the following prototype? Discuss.*

```
int f(int[] A);
```

Exercise 4 (sample solution on page 18)**.** *Consider the following function:*

```
int main() {
  int[] A = alloc_array(int, 3);
  A[0] = 1;
  A[1] = 2;
  A[2] = 2;
  int[] B = A;
  // memory diagram 1

  int[] C = alloc_array(int, 3);
  C[0] = A[0];
  C[1] = A[1];
  C[2] = A[2];
  // memory diagram 2

  B[2] = -1;
  C[2] = 1;
```

```
17   A = C;
18   B = alloc_array(int, 2);
19   // Memory diagram 3
20
21   return 0;
22 }
```

What are the value contained in the variable `B` *at line 7? Draw a memory diagram at this point of the execution. Next, extend your memory diagram to show what the memory looks like when execution reaches line 13. Finally, what are the values of* `A` *and* `B` *just before the function returns? Draw the final memory diagram. In each diagram, draw the contents of each array cell and mark garbage-collected memory (if any).*

Sample Solutions

Solution of exercise 1 We write a specification function `same_segs` that checks that two array segments of the same length contain the same value in each corresponding position.

```
bool same_segs(int[] A, int loA, int hiA,
               int[] B, int loB, int hiB)
//@requires 0 <= loA && loA <= hiA && hiA <= \length(A);
//@requires 0 <= loB && loB <= hiB && hiB <= \length(B);
//@requires hiA - loA == hiB - loB;
{
  int i = loA;
  int j = loB;
  while (i < hiA)
  //@loop_invariant loA  <= i && i <= hiA;
  //@loop_invariant loB  <= j && j <= hiB;
  //@loop_invariant hiA - i == hiB - j;
  {
    //@assert j < hiB;
    if (A[i] != B[j])
      return false;
    i++;
    j++;
  }
  return true;
}
```

The function `array_part` creates an array of length `hi-lo` and copies the values of the segment `A[lo,hi)` into it. Besides the usual loop invariant for the loop index, we have a second loop invariant that uses `same_segs` to state that, at each iteration of the loop, the new array contains the same values as `A` up to this point.

```
30 int[] array_part(int[] A, int lo, int hi)
31 //@requires 0 <= lo && lo <= hi && hi <= \length(A);
32 //@ensures same_segs(A, lo, hi, \result, 0, hi-lo);
33 {
34   int[] res = alloc_array(int, hi-lo);
35   for (int i = lo; i < hi; i++)
36   //@loop_invariant lo <= i && i <= hi;
37   //@loop_invariant same_segs(A, lo, i, res, 0, i-lo);
38   {
39     res[i-lo] = A[i];
40   }
41   return res;
42 }
```

Before we prove the correctness of `array_part`, we will prove that the loop invariants of the specification function `same_segs` are valid (INIT + PRES). Along the way we will prove that the assertion in the loop body is true. We conclude with showing that this loop terminates (TERM).

INIT We need to show that both loop invariants (on line 10, line 11 and line 12) are true initially:

To Show: `loA <= i && i <= hiA` is true initially

a.	`i = loA`	by line 7
b.	`loA <= i`	by math on (a)
c.	`loA <= hiA`	by line 3
d.	`i <= hiA`	by math on (c) and (a)

To Show: `loB <= j && j <= hiB` is true initially

a.	`j = loB`	by line 8
b.	`loB <= j`	by math on (a)
c.	`loB <= hiB`	by line 4
d.	`j <= hiB`	by math on (c) and (a)

To Show: `hiA - i == hiB - j` is true initially

a.	`i = loA`	by line 7
b.	`j = loB`	by line 8
c.	`hiA - loA == hiB - loB`	by line 5
d.	`hiA - i == hiB - j`	by math on (a), (b) and (c)

Assertion on line 14 (assuming the loop invariants are valid)

To Show: `j < hiB`

a.	`i < hiA`	by line 9
b.	`hiA - i > 0`	by math on (a)
c.	`hiA - i == hiB - j`	by line 12
d.	`hiB - j > 0`	by math on (b) and (c)
e.	`j < hiB`	by math on (d)

PRES We need to show that the loop invariants (on line 10, line 11 and line 12) are preserved by an arbitrary iteration of the loop:

Assume: `loA <= i && i <= hiA`

To Show: `loA <= i' && i' <= hiA`

a.	`i' = i+1`	by line 17
b.	`loA <= i`	by assumption
c.	`i < hiA`	by line 9
d.	`loA <= i+1`	by math on (b) and (c)
e.	`i+1 <= hiA`	by math on (c)
f.	`loA <= i' && i' <= hiA`	by (a), (d) and (e)

Assume: `loB <= i && i <= hiB`

To Show: `loB <= j' && j' <= hiB`

a.	`j' = j+1`	by line 18
b.	`loB <= j`	by assumption
c.	`j < hiB`	by line 14
d.	`loB <= j+1`	by math on (b) and (c)
e.	`j+1 <= hiB`	by math on (c)
f.	`loB <= j' && j' <= hiB`	by (a), (d) and (e)

Assume: `hiA - i == hiB - j`

To Show: `hiA - i' == hiB - j'`

a.	`i' = i+1`	by increment on line 17
b.	`j' = j+1`	by increment on line 18
c.	`hiA - (i+1) == hiB - (j+1)`	by math on assumption
d.	`hiA - i' == hiB - j'`	by math on (a), (b) and (d)

TERM At each iteration of the loop, the expression `hiA-i` is strictly decreasing (because `i` is incremented while `hiA` stays constant) and can never become less then `0`. Moreover the loop guard is false when `hiA-i == 0`.

Next, we prove the correctness of `array_part`. As usual for a function with one loop, we do so by showing the the loop invariants are valid (INIT + PRES), that the postconditions are met (EXIT), and that the loop terminates (TERM).

INIT We need to show that both loop invariants (on line 36 and line 37) are true initially:

To Show: `lo <= i && i <= hi` is true initially

a.	`i = lo`	by line 35
b.	`lo <= i`	by math on (a)
c.	`lo <= hi`	by line 31
d.	`i <= hi`	by math on (c) and (a)

To Show: `same_segs(A,lo,i, res,0,hi-i)` is true initially

a.	`i == lo`	by line 35
b.	`lo - lo == 0`	by math on (a)
c.	`same_segs(A,lo,lo, res,0,0)`	by definition of `same_segs`

PRES We need to show that both loop invariants (on line 36 and line 37) are preserved by an arbitrary iteration of the loop:

Assume: `lo <= i && i <= hi`

To Show: `lo <= i' && i' <= hi`

a.	`i' = i+1`	by increment on line 35
b.	`lo <= i`	by assumption
c.	`i < hi`	by loop guard on line 35
d.	`lo <= i+1`	by math on (b) and (c)
e.	`i+1 <= hi`	by math on (c)
f.	`lo <= i' && i' <= hi`	by (a), (d) and (e)

Assume: `same_segs(A,lo,i, res,0,i-lo)`

To Show: `same_segs(A,lo,i', res,0,i'-lo)`

a.	`x' = x+1`	by increment on line 35

b.	`res[i-lo] == A[i]`	by line 39
c.	`same_segs(A,lo,i‘, res,0,i‘-lo)`	by assumption and (b)

EXIT We need to show that the postcondition on line 32 is met.

To Show: `same_segs(A, lo, hi, res, 0, hi-lo)`

a.	`i >= hi` on line 35	by negation of loop guard
b.	`i <= hi`	by line 36
c.	`i == hi`	by math on (a) and (b)
d.	`same_segs(A,lo,i, res,0,i-lo)`	by line 37
e.	`same_segs(A,lo,hi, res,0,hi-lo)`	by math on (c) and (d)

TERM At each iteration of the loop, the expression `hi-i` is strictly decreasing (because `i` is incremented while `hi` stays constant) and can never become less then `0`. Moreover the loop guard is false when `hi-i == 0`.

Solution of exercise 2 We can use the function `same_seg` developed in exercise 1 as the requested specification function. Then, the code for `copy_into` is as follows:

```
50 int copy_into(int[] source, int i, int n, int[] target, int j)
51 //@requires n >= 0;
52 //@requires 0 <= i && i+n <= \length(source);
53 //@requires 0 <= j && j+n <= \length(target);
54 //@ensures same_segs(source, i, i+n, target, j, j+n);
55 //@ensures (n == 0 && \result == -1) || (n > 0 && \result == j+n-1);
56 {
57   for (int x = 0; x < n; x++)
58   //@loop_invariant 0 <= x && x <= n;
59   //@loop_invariant same_segs(source, i, i+x, target, j, j+x);
60   {
61     target[j+x] = source[i+x];
62   }
63   if (n == 0)
64     return -1;
65   return j + n - 1;
66 }
```

The proof of correctness proceeds by showing the the loop invariants are valid (INIT + PRES), that the postconditions are met (EXIT), and that the loop terminates (TERM).

INIT We need to show that both loop invariants (on line 58 and line 59) are true initially:

To Show: `0 <= x && x <= n` is true initially

a.	`x = 0`	by line 57
b.	`x >= 0`	by math on (a)
c.	`n >= 0`	by line 51
d.	`x <= n`	by math on (c) and (a)

To Show: `same_segs(source, i, i+x, target, j, j+x)` is true initially

a.	`x == 0`	by line 57
b.	`same_segs(source, i, i, target, j, j)`	by definition of `same_segs`

PRES We need to show that both loop invariants (on line 58 and line 59) are preserved by an arbitrary iteration of the loop:

Assume: `0 <= x && x <= n`

To Show: `0 <= x' && x' <= n`

a.	`x' = x + 1`	by increment on line 57
b.	`x >= 0`	by assumption
c.	`x < n`	by loop guard on line 57
d.	`x + 1 >= 0`	by math on (b) and (c)
e.	`x + 1 <= n`	by math on (c)
f.	`0 <= x' && x' <= n`	by (a), (d) and (e)

Assume: `same_segs(source, i, i+x, target, j, j+x)`

To Show: `same_segs(source, i, i+x', target, j, j+x')`

a.	`x' = x + 1`	by increment on line 57
b.	`target[j+x] == source[i+x]`	by line 61
c.	`same_segs(source, i, i+x', target, j, j+x')`	by assumption and (b)

EXIT We need to show that the postconditions on lines 54 and 55 are met.

To Show: `same_segs(source, i, i+n, target, j, j+n)`

a.	`x >= n`	by negation of loop guard on line 57
b.	`x <= n`	by line 58

c. `x = n` by math on (a) and (b)

d. `same_segs(source, i, i+x, target, j, j+x)` by line 59

e. `same_segs(source, i, i+n, target, j, j+n)` by math on (c) and (d)

To Show: `(n == 0 && \result == -1) || (n > 0 && \result == j+n-1)`

We distinguish cases based on where the function returns:

- If the function returns on line 64:

  a. `n == 0` by line 63

  b. `\result== -1` by line 64

  c. `n == 0 && \result== -1` by (a) and (b)

- If the function returns on line 65:

  a. `n > 0` by lines 63 and 51

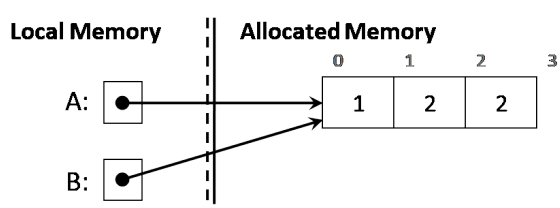
  b. `\result== j+n-1` by line 65

  c. `n > 0 && \result== j+n-1` by (a) and (b)

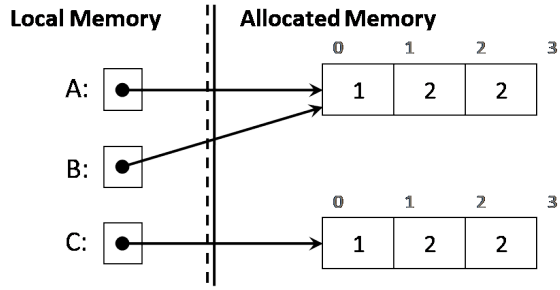
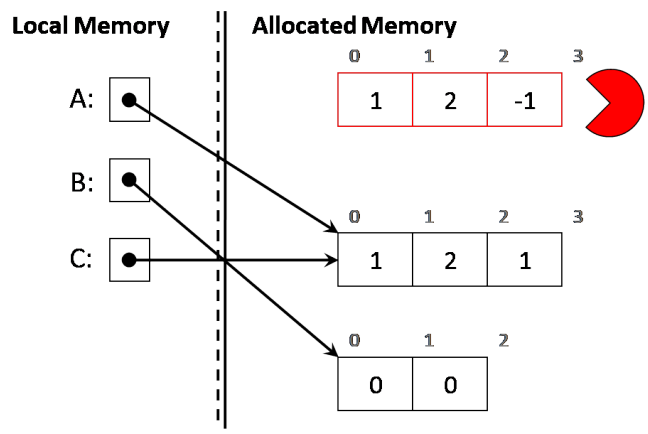
TERM At each iteration of the loop, the expression `n-x` is strictly decreasing (because `x` is incremented while `n` stays constant) and can never become less then `0`. Moreover the loop guard is false when `n-x == 0`.

Solution of exercise 3 There are not such functions that you could create with the following prototype. At first glance, it seems that you could return an integer that gives information about the array. However, we are not given a parameter for the length of the array. We might want to access `A[i]`, but without the length, we are not guaranteed that the length of `A` is greater than `i`. Since we cannot access any of the elements safely, there is no information that you could get from the array.

Solution of exercise 4 At line 7, the variable `B` contains the same address as the variable `A` — they are aliases. The memory diagram is as follows:



Once execution reaches line 13, the contents of memory are as follows



Just before the end of the function (line 19), A contains the address of the array allocated at line 9 and B contains the address of the array allocated at line 18. Note that the array allocated at line 2 is not accessible at this point. Therefore it is marked for garbage collection. The memory diagram is as follows: